Advaitum Business OS

Free Launch Guide · GitHub + Cloudflare Pages + Supabase

Version 1.0 · 29 October 2026

Goal: launch a responsive Chrome-based business app at ■0 upfront, within free-plan limits.

Read this first. The supplied starter is a UI demo only. It uses in-memory sample data and is not connected to Supabase. Do not use it for real inventory or financial records until authentication, RLS, transaction-safe stock posting, costing tests and backup/restore are complete.

Recommended ■0 starter stack

• GitHub Free — private source repository.

• Cloudflare Pages — static frontend hosting with a free pages.dev address.

• Supabase Free — PostgreSQL and authentication within free-plan limits.

• Node.js LTS + Git + VS Code — local development.

Vercel Hobby is restricted to personal, non-commercial use. For an Advaitum business tool, use a host whose current terms permit the intended use; verify current plan terms before launch.

Phase 1 — Install and run locally

Install Node.js LTS, Git and VS Code. Extract the ZIP and open the advaitum-app folder in VS Code. Run:

```
node -v\nnpm -v\ngit --version\nnpm install\nnpm run dev
```

Open http://localhost:3000 in Chrome. Stop the server with Ctrl+C.

Phase 2 — Push to private GitHub

Create a private repository named advaitum-business-os at https://github.com/new. Do not initialize with a README. Run from the project folder:

```
git init\ngit add .\ngit commit -m "Initial Advaitum Business OS starter"\ngit branch -M main\ngit remote add origin https:/
```

Replace YOUR_GITHUB_USERNAME. Never commit .env.local, passwords, tokens or a Supabase service-role/secret key.

Phase 3 — Create Supabase project

• Create a Free project at https://supabase.com/dashboard and save the database password securely.

• Get the Project URL and publishable key (older projects may call it the anon key). Never use service-role/secret keys in browser code.

• Review supabase/migrations/0001_initial_schema.sql in a development project first.

• The current schema enables RLS but does not yet contain all required policies. Do not expose business tables until least-privilege policies are implemented and tested.

• Free projects may pause after inactivity and do not include all backup/recovery features; create a regular export and test restore.

Phase 4 — Configure static export

The current starter is client-side and has no Next.js server routes. For this starter, static export is the simplest Cloudflare Pages route. Replace next.config.ts with:

```
import type { NextConfig } from 'next';\n\nconst nextConfig: NextConfig = {\n  output: 'export',\n  trailingSlash: true,\n};
```

Run npm run build and confirm an out/ folder is generated. Commit and push the change. If server-only features are added later, reassess hosting architecture.

Phase 5 — Deploy to Cloudflare Pages

• Open https://dash.cloudflare.com/ → Workers & Pages → Create → Pages → connect to Git.

• Connect GitHub and select advaitum-business-os.

• Build command: npm run build. Output directory: out.

• If the repository root is the app folder, leave root directory blank. If the app is nested in a parent repository, set root directory to advaitum-app.

• Deploy and open the resulting *.pages.dev URL in desktop and mobile Chrome.

• This proves the UI is hosted only; it does not mean Supabase persistence or security is complete.

Phase 6 — Connect Supabase safely

Use .env.example as the local template. Create .env.local locally and fill NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY. Keep .env.local out of Git. Add the same public values to Cloudflare Pages environment settings only after the Supabase client and persistence code are implemented. RLS policies—not secrecy of the public key—must protect database access.

Phase 7 — Build features in this order

• Authentication and owner-only access.

• Products/categories CRUD backed by Supabase.

• Purchases, suppliers, purchase lots and landed-cost allocation.

• Append-only stock movement ledger.

• Atomic sales posting that writes sale, line items, COGS and stock movements in one database transaction.

• Choose and test weighted-average or FIFO costing; never mix silently.

• Returns, cancellations, damaged stock and adjustments as reversing ledger entries.

• Date/SKU/category/channel/location reports and CSV export.

• Validated import, duplicate detection, audit trail, backup and tested restore.

• Mobile/laptop QA and permission tests.

Calculation definitions

• Gross profit = net product revenue − COGS.

• Gross margin % = gross profit / net product revenue × 100; show — if revenue is zero.

• Contribution = gross profit − directly attributable channel fees, shipping, packaging and other selling costs.

• Stock on hand = opening stock + receipts + returns − sales − damage/adjustments out.

• Inventory value must use the chosen costing method.

Production go-live checklist

• ■ Unauthenticated visitors cannot read business records.

- ■ RLS policies and grants are tested.
- ■ No secret keys are committed or exposed to browser code.
- ■ Sales posting is atomic and blocks accidental overselling.
- ■ Returns/cancellations reverse stock and money correctly.
- ■ Reports reconcile with ledgers and exports.
- ■ Backup/export and restore tested in a separate project.
- ■ Mobile and desktop layouts, errors and empty states tested.
- ■ Only then import real records.

Official documentation

- **Node.js:** https://nodejs.org/
- **Git:** https://git-scm.com/downloads
- **GitHub docs:** https://docs.github.com/
- **Next.js static export:** https://nextjs.org/docs/app/guides/static-exports
- **Cloudflare Pages:** https://developers.cloudflare.com/pages/
- **Cloudflare Pages Git integration:** https://developers.cloudflare.com/pages/configuration/git-integration/
- **Supabase pricing:** https://supabase.com/pricing
- **Supabase Row Level Security:** https://supabase.com/docs/guides/database/postgres/row-level-security
- **Supabase JavaScript client:** https://supabase.com/docs/reference/javascript/introduction